



使用JDK网络基础组件编写简单网络应用

北京理工大学计算机学院
金旭亮

JDK中的网络组件



JDK中的`java.net`包提供了一些网络组件，比如，`URL`和`URLConnection`等类提供了以编程方式访问Web网站的功能，而`URLDecoder`和`URLEncoder`则提供了普通字符串和“`application/x-www-form-urlencoded`” MIME 字符串相互转换的静态方法。



IP地址由`InetAddress`类来代表，`InetAddress`下还有两个子类：`Inet4Address`、`Inet6Address`，它们分别代表Internet Protocol version 4 (IPv4) 地址和Internet Protocol version 6 (IPv6) 地址。

InetAddress



代表IP地址，主要用于创建Socket。

```
private static InetAddress useInteAddress() throws IOException {  
    // 根据主机名来获取对应的InetAddress实例  
    InetAddress ip = InetAddress.getByName(host: "www.jinxuliang.com");  
    // 判断是否可达  
    System.out.println("教学网站是否可达:" + ip.isReachable(timeout: 2000));  
    // 获取该InetAddress实例的IP字符串  
    System.out.println(ip.getHostAddress());  
    // 根据原始IP地址来获取对应的InetAddress实例  
    InetAddress local = InetAddress.getByAddress(  
        new byte[]{127, 0, 0, 1});  
    System.out.println("本机是否可达:" + local.isReachable(timeout: 5000));  
    // 获取该InetAddress实例对应的全限定域名  
    System.out.println(local.getCanonicalHostName());  
    return ip;  
}
```

InetAddressTest.java



```
Run: InetAddressTest x  
"C:\Program Files\Ja  
教学网站是否可达:true  
182.92.1.231  
本机是否可达:true  
127.0.0.1  
Process finished wit
```

InetSocketAddress

InetSocketAddress = InetAddress + Port

```
private static void useInetSocketAddress() throws UnknownHostException {  
    InetAddress ip = InetAddress.getByName(host: "www.jinxuliang.com");  
    var socketAddress = new InetSocketAddress(ip, port: 9999);  
    //www.jinxuliang.com/182.92.1.231:9999  
    System.out.println(socketAddress);  
    //www.jinxuliang.com  
    System.out.println(socketAddress.getHostName());  
    //www.jinxuliang.com/182.92.1.231  
    System.out.println(socketAddress.getAddress());  
    //9999  
    System.out.println(socketAddress.getPort());  
    //www.jinxuliang.com  
    System.out.println(socketAddress.getHostString());  
    //false  
    System.out.println(socketAddress.isUnresolved());  
}
```

InetSocketAddress封装了IP地址和端口信息，它本质上就是Socket的地址信息。

获取本机地址



```
public class MyAddress {  
    public static void main(String[] args)  
        throws UnknownHostException {  
        var address :InetAddress = InetAddress.getLocalHost();  
        System.out.println("Name:" + address.getCanonicalHostName());  
        System.out.println("IP:" + address.getHostAddress());  
    }  
}
```

MyAddress.java

运行结果

```
Run: MyAddress x  
▶ "C:\Program Files\Java\jdk-17\bin\java.exe"  
Name:DESKTOP-DQ7I8UF  
IP:192.168.0.109  
⏏  
Process finished with exit code 0
```

网络接口



NetworkInterface 类包含网络接口名称与IP地址列表。



本台机上的所有NetworkInterface，可以通过以下方法获取：

```
NetworkInterface.getNetworkInterfaces()
```

获取本机的所有网络接口

```
private static void listAllNetworkInterface() throws SocketException {  
    var networks : Enumeration<NetworkInterface> = NetworkInterface.getNetworkInterfaces();  
    while (networks.hasMoreElements()) {  
        NetworkInterface network = networks.nextElement();  
        System.out.println("接口名(短名) : " + network.getName());  
        System.out.println("接口名(长名) : " + network.getDisplayName());  
        var addresses : Enumeration<InetAddress> = network.getInetAddresses();  
        while (addresses.hasMoreElements()) {  
            var address : InetAddress = addresses.nextElement();  
            if (address instanceof Inet4Address) {  
                System.out.printf("%-6s %-15s\n", " IPv4:", address);  
            } else {  
                System.out.printf("%-6s %-20s\n", " IPv6:", address);  
            }  
        }  
        System.out.println();  
    }  
}
```

NetworkInterfaceInfo.java

运行截图

```
Run: NetworkInterfaceInfo x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
接口名(短名) : lo
接口名(长名) : Software Loopback Interface 1
    IPv4: /127.0.0.1
    IPv6: /0:0:0:0:0:0:0:0:1

接口名(短名) : eth0
接口名(长名) : WAN Miniport (Network Monitor)

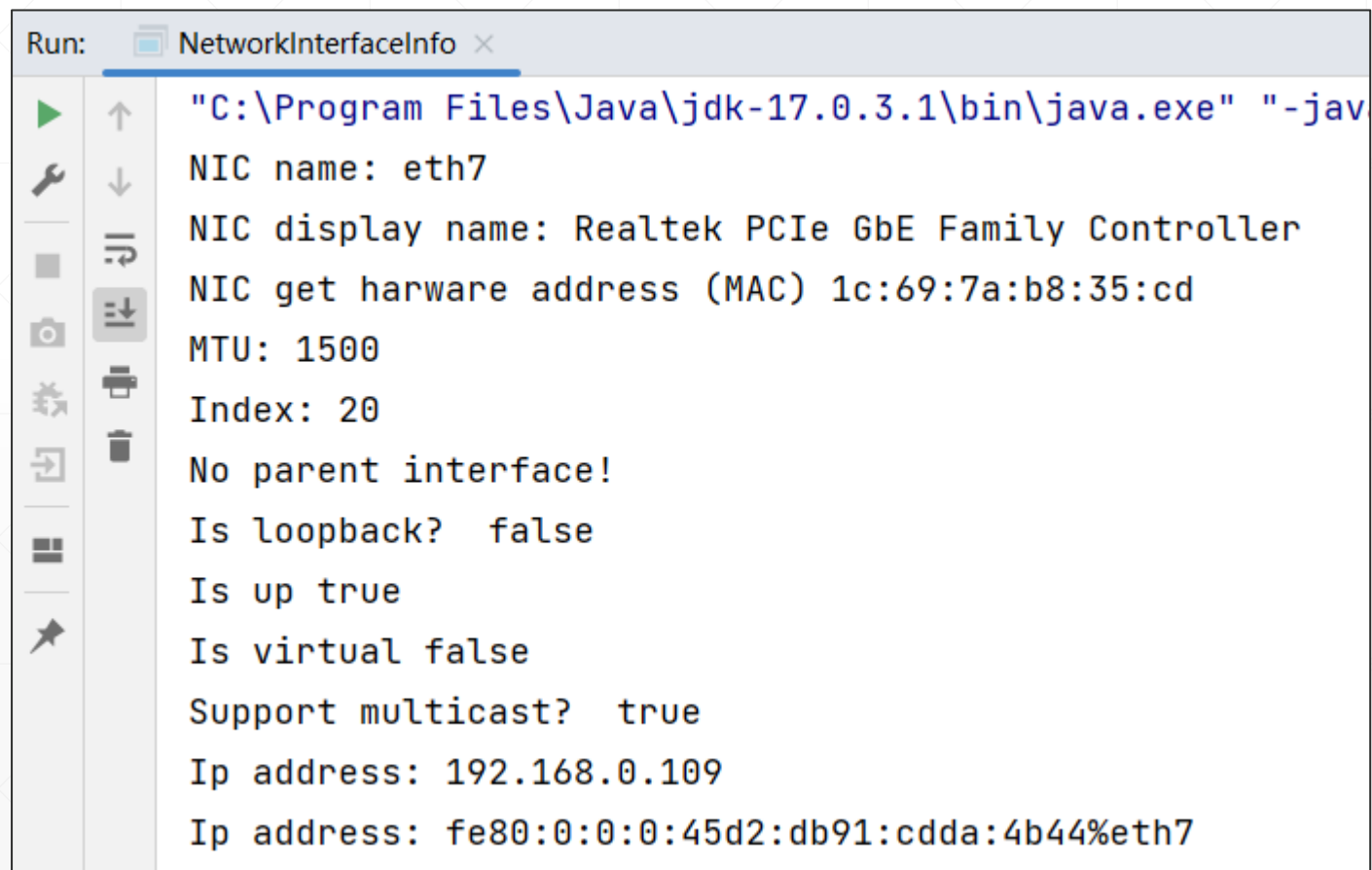
接口名(短名) : net0
接口名(长名) : Microsoft 6to4 Adapter
```

```
Structure
NetworkInterface
  static class initializer jdk.internal.loader....
  inetAddresses(): Stream<InetAddress>
  subInterfaces(): Stream<NetworkInterface>
  getByName(String): NetworkInterface
  getByIndex(int): NetworkInterface
  getByInetAddress(InetAddress): NetworkInterface
  networkInterfaces(): Stream<NetworkInterface>
  supportsMulticast(): boolean
  equals(Object): boolean ↑Object
  hashCode(): int ↑Object
  toString(): String ↑Object
  > p name: String
  > p inetAddresses: Enumeration<InetAddress>
  > p interfaceAddresses: List<InterfaceAddress>
  > p subInterfaces: Enumeration<NetworkInterface>
  > p parent: NetworkInterface
  > p index: int
  > p displayName: String
  > p networkInterfaces: Enumeration<NetworkInterface>
  > p up: boolean
  > p loopback: boolean
  > p pointToPoint: boolean
  > p hardwareAddress: byte[]
  > p MTU: int
  > p virtual: boolean
```

NetworkInterface这个类中包容有相当丰富的信息，请仔细看它的成员，在实际开发中相当有用。

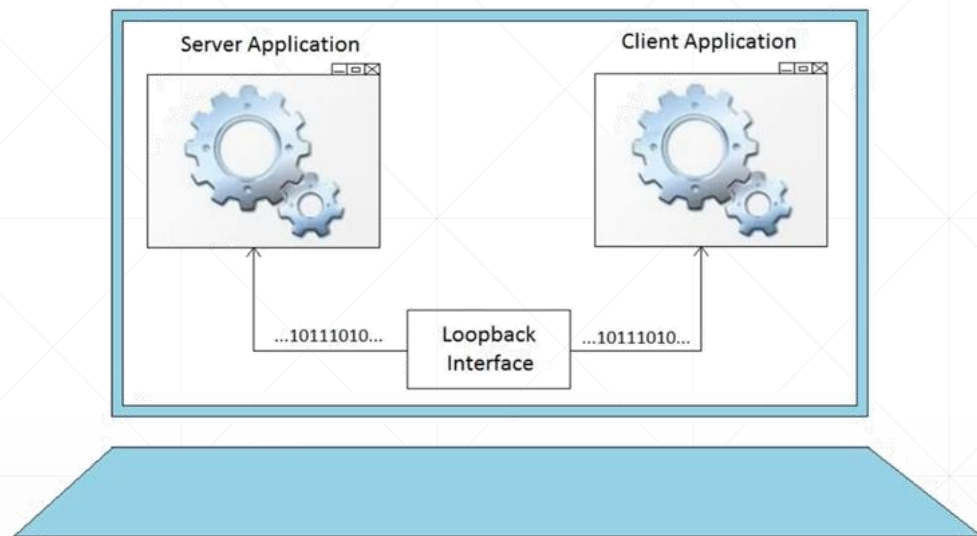
查看特定网卡的详细参数

参看示例中的printNetworkInfo()方法的输出：



```
Run: NetworkInterfaceInfo x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" "-jav
NIC name: eth7
NIC display name: Realtek PCIe GbE Family Controller
NIC get hardware address (MAC) 1c:69:7a:b8:35:cd
MTU: 1500
Index: 20
No parent interface!
Is loopback? false
Is up true
Is virtual false
Support multicast? true
Ip address: 192.168.0.109
Ip address: fe80:0:0:0:45d2:db91:cdda:4b44%eth7
```

一种特殊的网络接口—Loopback



它其实是一个“虚拟网卡”，发往它的数据包并没有真正到网络中，而只是在“本机逛了一圈”。

在实际开发中可以使用它来测试网络应用程序是否工作正常。

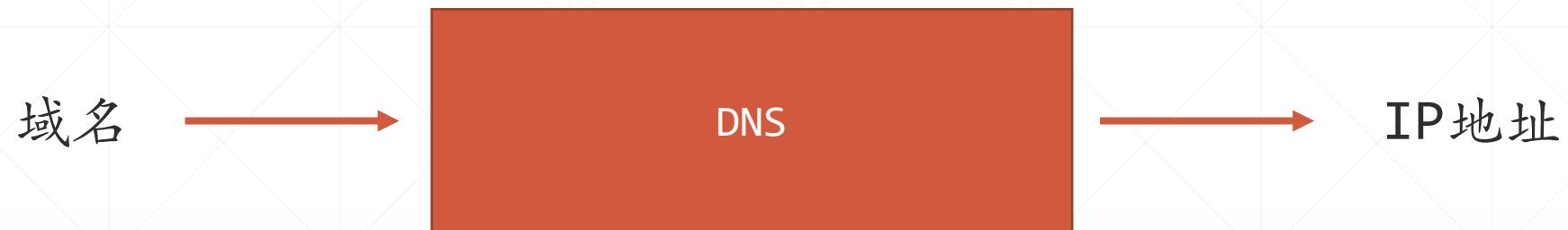
NetworkInterface中的一个重要方法

`public boolean isUp()`方法：判断网络接口是否已经开启并正常工作。



被禁用或网线被拔掉时，此方法返回false。

DNS: Domain Name System



Java编程获取域名对应的IP地址



```
public class DNSDemo {  
    public static void main(String[] args) {  
        System.out.println("输入域名: ");  
        var input = new BufferedReader(new InputStreamReader(System.in));  
        input.lines().forEach(line -> {  
            try {  
                var address : InetAddress = InetAddress.getByName(line);  
                System.out.println("IP:" + address.getHostAddress());  
            } catch (UnknownHostException e) {  
                System.out.println(e.getMessage());  
            }  
        });  
    }  
}
```

DNSDemo.java

```
Run: DNSDemo x  
"C:\Program Files\J  
输入域名:  
www.baidu.com  
IP:110.242.68.4  
jinxuliang.com  
IP:182.92.1.231  
|
```

向指定主机发出ping请求



TestPing.java

```
public static void main(String[] args) {  
    try {  
        String hostAddress = "www.baidu.com";  
        InetAddress host = InetAddress.getByName(hostAddress);  
  
        Process p = Runtime.getRuntime().exec("ping " + hostAddress);  
  
        var inputStream = new BufferedReader(  
            new InputStreamReader(p.getInputStream(),  
                Charset.forName("GBK")));  
        String commandOutput = "";  
        while ((commandOutput = inputStream.readLine()) != null) {  
            System.out.println(commandOutput);  
        }  
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
}
```

```
Run TestPing x  
"C:\Program Files\Java\jdk-20\bin\java.exe" "-javaagent:C:\V  
正在 Ping www.a.shifen.com [110.242.68.3] 具有 32 字节的数据:  
来自 110.242.68.3 的回复: 字节=32 时间=11ms TTL=54  
来自 110.242.68.3 的回复: 字节=32 时间=11ms TTL=54  
来自 110.242.68.3 的回复: 字节=32 时间=10ms TTL=54  
来自 110.242.68.3 的回复: 字节=32 时间=10ms TTL=54  
  
110.242.68.3 的 Ping 统计信息:  
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),  
    往返行程的估计时间(以毫秒为单位):  
        最短 = 10ms, 最长 = 11ms, 平均 = 10ms  
  
Process finished with exit code 0
```

注意一下如何截获进程的输出信息。